

ПРЕЗЕНТАЦИЯ

АРХИТЕКТУРЫ БЭКЕНДА

Kanban AI Agent Backend

Дата: 24 октября 2025 г.

Версия: dev

СОДЕРЖАНИЕ

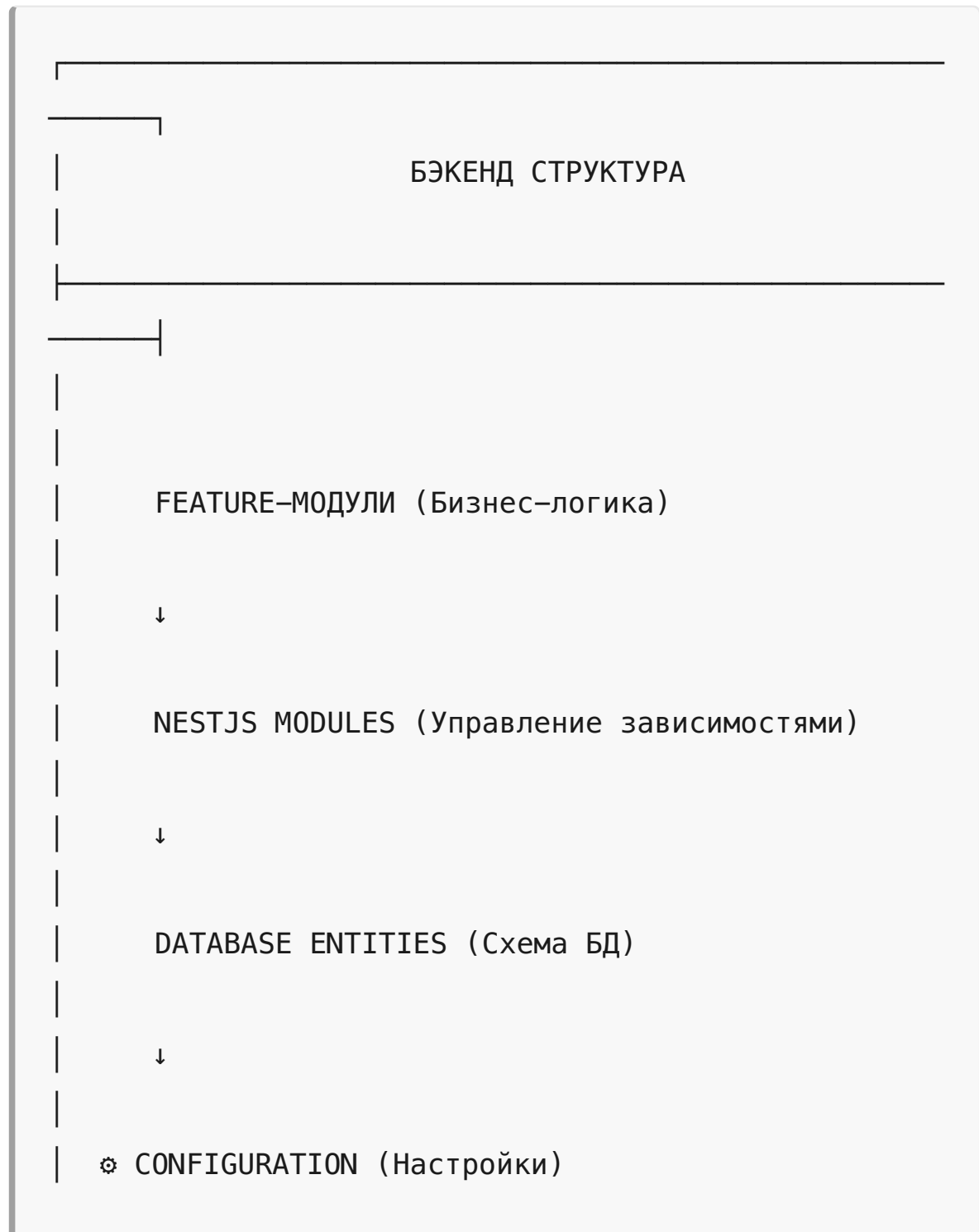
1. Общая архитектура
 2. Feature-модули
 3. Infrastructure-модули
 4. Configuration
 5. Database Entities
 6. NestJS Modules
 7. TypeScript Types
 8. Shared Services
 9. Как всё работает вместе
-

ОБЩАЯ АРХИТЕКТУРА

ЧТО ЭТО ЗА ПРОЕКТ?

Kanban AI Agent - это умная система управления задачами с AI автоматизацией

ОСНОВНЫЕ ЧАСТИ:



↓

SHARED SERVICES (Общие инструменты)

↓

TYPESCRIPT TYPES (Типизация)

ТЕХНОЛОГИИ:

- **NestJS** - фреймворк для Node.js
- **TypeScript** - строгая типизация
- **PostgreSQL** - основная база данных
- **Redis** - кэширование и очереди
- **BullMQ** - система очередей
- **TypeORM** - работа с БД
- **Claude AI** - искусственный интеллект

FEATURE-МОДУЛИ

Это основная бизнес-логика приложения. Каждый модуль отвечает за свою задачу.

1. KANBAN MANAGEMENT

ЧТО ЭТО?

Управление канбан-досками: задачи, колонки, комментарии, история

ДЛЯ ЧЕГО НУЖЕН?

Это основа всей системы - здесь живут все задачи, которые потом анализируют AI агенты

КАК РАБОТАЕТ?

ПОЛЬЗОВАТЕЛЬ СОЗДАЁТ ЗАДАЧУ

↓

POST /create-task

↓

ЗАДАЧА СОХРАНЯЕТСЯ В БД

↓

МОЖНО ПЕРЕМЕЩАТЬ МЕЖДУ КОЛОНКАМИ

↓

PATCH /move-task

↓

ИСТОРИЯ СОХРАНЯЕТСЯ АВТОМАТИЧЕСКИ

ЧТО УМЕЕТ?

Создавать/редактировать/удалять задачи

Перемещать между колонками (To Do → In Progress → Done)

Добавлять комментарии и файлы

Показывать всю историю изменений

Считать статистику (сколько задач, время выполнения)

Отслеживать активность пользователей

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `GET /get-tasks-by-column` - задачи в колонке
- `POST /create-task` - создать задачу
- `PATCH /update-task` - обновить задачу
- `PATCH /move-task` - переместить в другую колонку
- `GET /get-task-history` - история изменений
- `POST /add-comment` - добавить комментарий

С ЧЕМ РАБОТАЕТ?

Database → хранит все задачи

✂ **Cache** → быстрый доступ к частым задачам

Queue → асинхронные операции

Notifications → уведомления об изменениях

2. AI AGENT

ЧТО ЭТО?

Интеллектуальные агенты, которые автоматически выполняют задачи через AI

ДЛЯ ЧЕГО НУЖЕН?

Автоматизация работы с задачами. Создаёшь агента один раз, он работает постоянно.

КАК РАБОТАЕТ?

1. СОЗДАЁШЬ АГЕНТА

↓ **POST** /create-agent

```
{ name: "Task Analyzer", type: "analyzer" }
```

2. НАСТРАИВАЕШЬ ИНСТРУКЦИИ

↓

"Анализируй все новые задачи и добавляй теги"

3. ЗАПУСКАЕШЬ АГЕНТА

↓ **POST** /run-agent

4. АГЕНТ РАБОТАЕТ В ФОНЕ

↓ (через **Queue Management**)

- Берёт задачи из канбана
- Отправляет в **AI** (**Claude**)
- Получает результат
- Обновляет задачи

5. ПОЛУЧАЕШЬ РЕЗУЛЬТАТ

↓ **GET** /get-agent-result

```
{ status: "completed", tasksProcessed: 15 }
```

ЧТО УМЕЕТ?

Создавать агентов с кастомными инструкциями

Выполнять задачи автоматически

Запускаться по расписанию или вручную

Сохранять историю всех действий

Работать с контекстом (понимать связанные задачи)

Использовать разные AI модели

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /create-agent` - создать агента
- `POST /run-agent` - запустить на выполнение
- `GET /get-agent` - получить агента
- `GET /get-all-agents` - список всех агентов
- `GET /get-agent-result` - результат работы
- `PATCH /update-agent` - обновить настройки
- `DELETE /delete-agent` - удалить агента

С ЧЕМ РАБОТАЕТ?

Database → хранит агентов и инструкции

✂ **Cache** → кэширует результаты работы

Queue → выполняет агентов асинхронно

Context Management → управляет контекстом AI

Kanban Management → берёт и обновляет задачи

3. FLOW MANAGEMENT

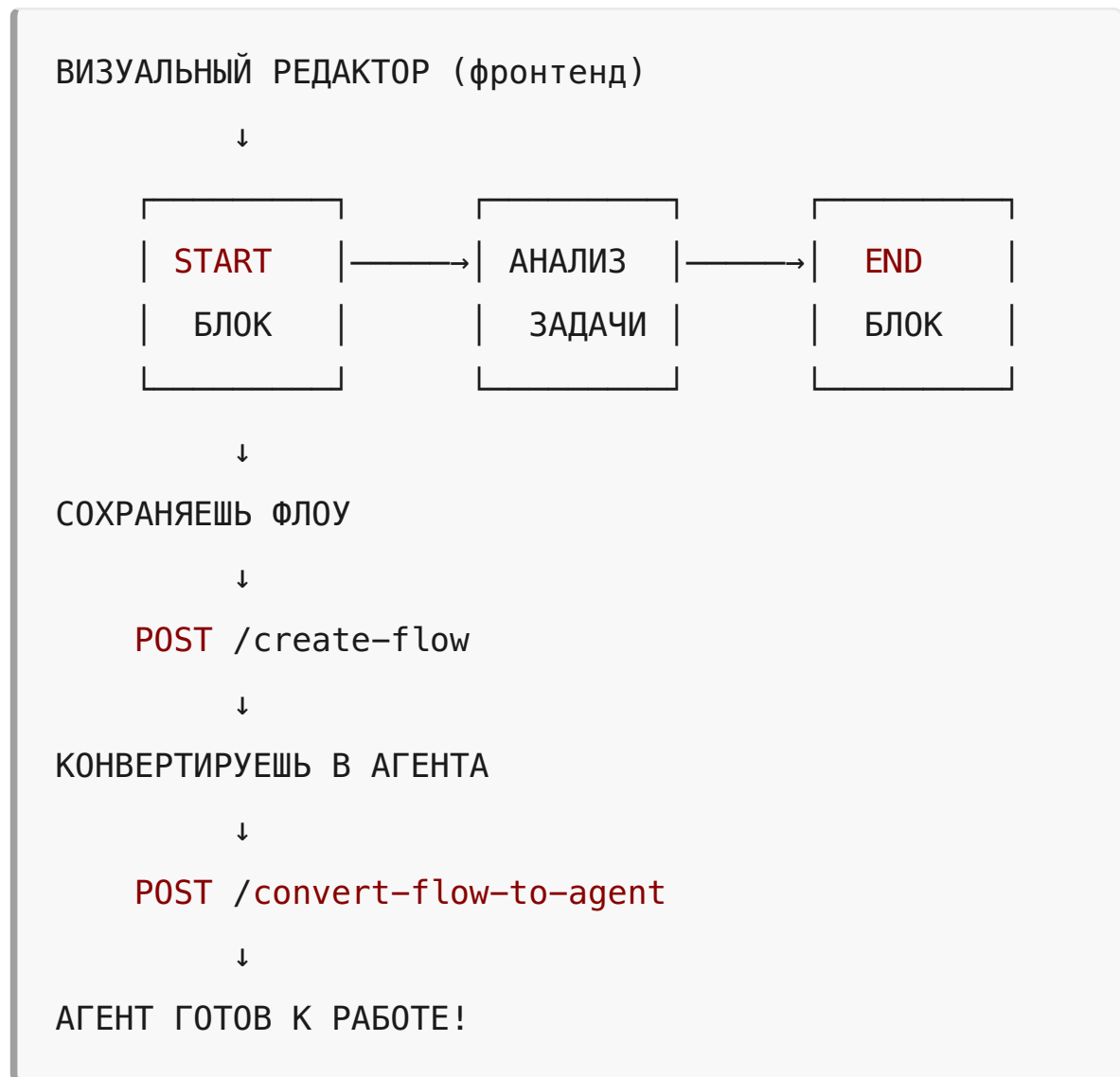
ЧТО ЭТО?

Визуальный конструктор потоков (flow builder) для создания автоматизации

ДЛЯ ЧЕГО НУЖЕН?

Вместо того чтобы писать код, ты визуально собираешь блоки → это превращается в AI агента

КАК РАБОТАЕТ?



ЧТО УМЕЕТ?

Создавать визуальные потоки из блоков

Сохранять и редактировать флоу

Конвертировать флоу в AI агента

Валидировать структуру (нет ли ошибок)

Показывать историю выполнения

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /create-flow` - создать флоу
- `GET /get-flow` - получить флоу
- `GET /get-all-flows` - список всех флоу
- `PATCH /update-flow` - обновить флоу
- `POST /convert-flow-to-agent` - превратить в агента
- `DELETE /delete-flow` - удалить флоу

С ЧЕМ РАБОТАЕТ?

Database → хранит флоу (блоки и связи)

↗ **Cache** → кэширует структуру

Queue → выполняет флоу асинхронно

Flow Conversion → конвертирует в агентов

AI Agent → создаёт агентов из флоу

4. JIRA INTEGRATION

ЧТО ЭТО?

Двусторонняя синхронизация с Jira - импорт и экспорт задач

ДЛЯ ЧЕГО НУЖЕН?

Команда работает в Jira? Не проблема! Синхронизируй задачи автоматически.

КАК РАБОТАЕТ?

1. ПОДКЛЮЧАЕШЬСЯ К JIRA

↓ POST /connect-jira

```
{ email: "you@company.com", apiToken: "xxx" }
```

2. ИМПОРТИРУЕШЬ ЗАДАЧИ

↓ POST /import-from-jira

```
{ projectKey: "PROJ", filter: "status=Open" }
```

3. СИСТЕМА СИНХРОНИЗИРУЕТ

↓

- Задачи из Jira → Канбан
- Статусы мапятся автоматически
- Комментарии переносятся

4. ЭКСПОРТИРУЕШЬ ОБРАТНО

↓ POST /export-to-jira

- Задачи из Канбана → Jira
- Сохраняется связь между системами

5. ДВУСТОРОННЯЯ СИНХРОНИЗАЦИЯ

↓ POST /sync-with-jira

- Изменения в Jira → обновляются в Канбане
- Изменения в Канбане → обновляются в Jira

ЧТО УМЕЕТ?

Подключаться к Jira (OAuth или API Token)

Импортировать задачи из Jira проектов

Экспортировать задачи обратно в Jira

Синхронизировать изменения автоматически

Маппить поля и статусы между системами

Работать с несколькими Jira проектами

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /connect-jira` - подключиться к Jira
- `GET /get-jira-connection` - статус подключения
- `GET /get-jira-projects` - список проектов
- `GET /get-jira-issues` - задачи из Jira
- `POST /import-from-jira` - импорт задач
- `POST /export-to-jira` - экспорт задач
- `POST /sync-with-jira` - синхронизация
- `DELETE /disconnect-jira` - отключиться

С ЧЕМ РАБОТАЕТ?

Database → хранит настройки подключения и маппинг

✂ **Cache** → кэширует данные из Jira

Kanban Management → создаёт и обновляет задачи

Notifications → уведомляет о синхронизации

Shared Services → использует базовый

BoardIntegrationService

5. AI REPORTING

ЧТО ЭТО?

Аналитика и отчёты по работе системы с использованием AI

ДЛЯ ЧЕГО НУЖЕН?

AI анализирует все задачи и даёт insights: что идёт не так, где узкие места, рекомендации.

КАК РАБОТАЕТ?

1. ЗАПРАШИВАЕШЬ ОТЧЁТ

↓ `POST /generate-report`

```
{ type: "productivity", period: "week" }
```

2. AI СОБИРАЕТ ДАННЫЕ

↓

- Все задачи за неделю
- История изменений
- Работа агентов
- Активность пользователей

3. AI АНАЛИЗИРУЕТ

↓ (через `Claude AI`)

- Паттерны в работе
- Проблемные области
- Метрики производительности

4. ГЕНЕРИРУЕТ ОТЧЁТ

↓

```
{  
  summary: "Команда выполнила 45 задач...",  
  insights: ["Задачи в колонке 'Review' висят  
дольше всего"],  
  recommendations: ["Добавить еще одного  
ревьюера"]  
}
```

5. ПОЛУЧАЕШЬ РЕЗУЛЬТАТ

↓ GET /get-report

ЧТО УМЕЕТ?

Генерировать отчёты через AI

Анализировать продуктивность команды

Показывать статистику по задачам

Анализировать работу AI агентов

Давать insights и рекомендации

Экспортировать отчёты в разных форматах

ОСНОВНЫЕ ЭНДПОИНТЫ:

- POST /generate-report - сгенерировать отчёт
- GET /get-report - получить отчёт
- GET /get-analytics-summary - сводка
- GET /get-productivity-stats - статистика

- `POST /analyze-tasks` - анализ задач
- `POST /get-insights` - получить insights

С ЧЕМ РАБОТАЕТ?

Database → берёт все данные для анализа

✂ **Cache** → кэширует готовые отчёты

Kanban Management → данные по задачам

AI Agent → данные по работе агентов

Context Management → контекст для AI

6. PHOTO ANALYSIS

ЧТО ЭТО?

Анализ изображений и фотографий через AI

ДЛЯ ЧЕГО НУЖЕН?

Загружаешь скриншот или фото → AI создаёт задачу автоматически!

КАК РАБОТАЕТ?

1. ЗАГРУЖАЕШЬ ФОТО

↓ `POST /upload-and-analyze`

(загружаешь скриншот бага из приложения)

2. AI АНАЛИЗИРУЕТ

↓ (через `Claude Vision API`)

- Распознаёт объекты на изображении
- Читает текст (`OCR`)
- Понимает контекст

3. AI ГЕНЕРИРУЕТ ОПИСАНИЕ

↓

```
{  
  title: "Кнопка 'Сохранить' смещена вправо",  
  description: "На странице настроек кнопка...",  
  tags: ["UI", "bug", "design"],  
  priority: "medium"  
}
```

4. СОЗДАЁТ ЗАДАЧУ

↓ `POST /create-task-from-photo`

Задача автоматически добавляется в канбан!

ЧТО УМЕЕТ?

Загружать и анализировать изображения

Распознавать объекты на фото

Извлекать текст (OCR)

Генерировать описания через AI

Классифицировать изображения

Автоматически создавать задачи из фото

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /analyze-photo` - анализ фото
- `POST /upload-and-analyze` - загрузка и анализ
- `POST /create-task-from-photo` - создать задачу
- `GET /get-analysis-result` - результат анализа

С ЧЕМ РАБОТАЕТ?

Database → хранит результаты анализа

✂ **Cache** → кэширует анализ повторяющихся фото

Kanban Management → создаёт задачи

Context Management → контекст для AI

AI Agent → может использоваться в агентах

7. BOARD INTEGRATIONS

ЧТО ЭТО?

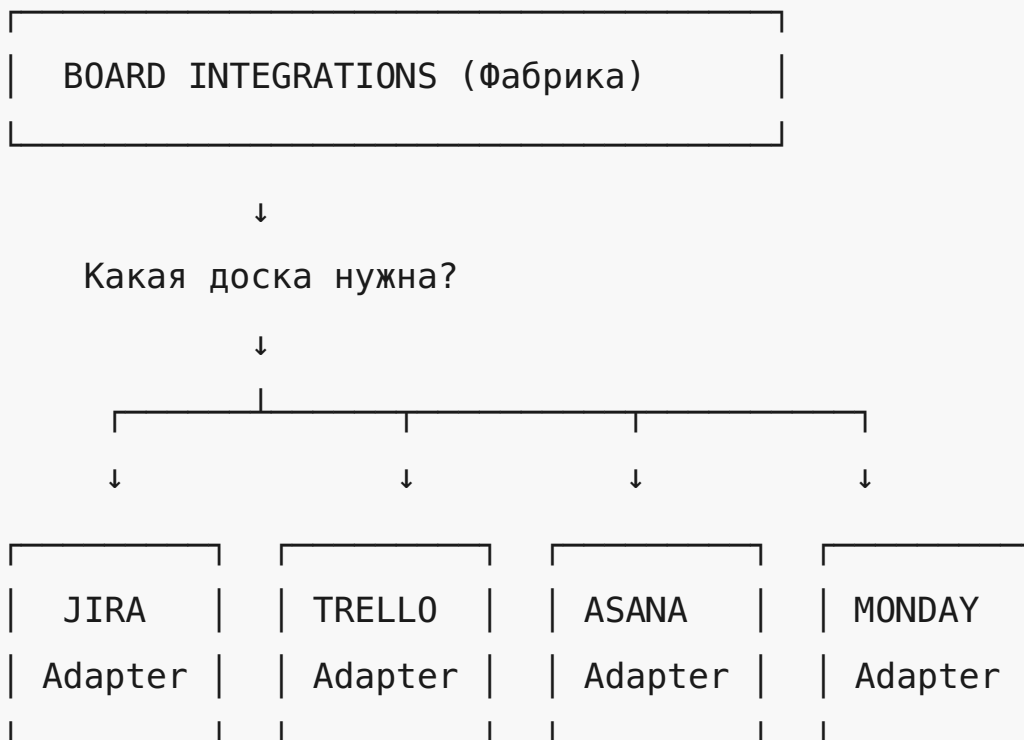
Интеграции с различными системами управления задачами

ДЛЯ ЧЕГО НУЖЕН?

Работает не только с Jira, но и с Trello, Asana, Monday, Linear и другими!

КАК РАБОТАЕТ?

УНИВЕРСАЛЬНЫЙ ПОДХОД:



ВСЕ АДАПТЕРЫ ИМЕЮТ ОДИНАКОВЫЕ МЕТОДЫ:

- connect()
- getTasks()
- createTask()
- syncTasks()

ЧТО УМЕЕТ?

Подключаться к разным платформам

Унифицированный API для всех интеграций

Двусторонняя синхронизация

Автоматический маппинг полей

Разрешение конфликтов при синхронизации

Batch операции (массовый импорт)

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /connect-board` - подключить доску
- `GET /get-integrations` - список интеграций
- `GET /get-integration-status` - статус
- `POST /sync-board` - синхронизировать
- `POST /import-from-board` - импорт
- `DELETE /disconnect-board` - отключить

С ЧЕМ РАБОТАЕТ?

Database → хранит настройки всех интеграций

✂ **Cache** → кэширует данные из внешних систем

Kanban Management → создаёт задачи

Queue → асинхронная синхронизация

Shared Services → использует BaseBoardIntegrationService

8. FLOW CONVERSION

ЧТО ЭТО?

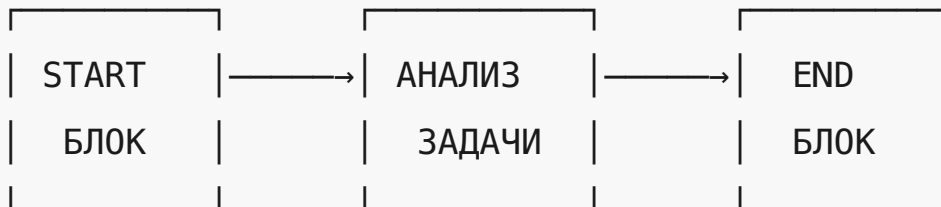
Конвертация визуальных флоу в исполняемых AI агентов

ДЛЯ ЧЕГО НУЖЕН?

Это "мозг", который превращает визуальные блоки в инструкции для AI.

КАК РАБОТАЕТ?

ВИЗУАЛЬНЫЙ ФЛОУ:



↓ КОНВЕРТАЦИЯ

ПРОМПТ ДЛЯ AI АГЕНТА:

"Ты – AI агент для анализа задач."

1. Получи список задач
2. Проанализируй каждую задачу
3. Добавь теги на основе содержимого
4. Верни результат"

↓

ГОТОВЫЙ AI АГЕНТ!

ЧТО УМЕЕТ?

Преобразовывать блоки флоу в инструкции
Валидировать структуру флоу
Оптимизировать последовательность блоков
Генерировать промпты для AI
Создавать связи между блоками
Поддерживать условную логику (if/else)

С ЧЕМ РАБОТАЕТ?

Database → хранит флоу и агентов
AI Agent → создаёт агентов
Flow Management → основная логика конвертации
AI Reporting → статистика конвертаций

⚡ INFRASTRUCTURE-МОДУЛИ

9. DATABASE MANAGEMENT

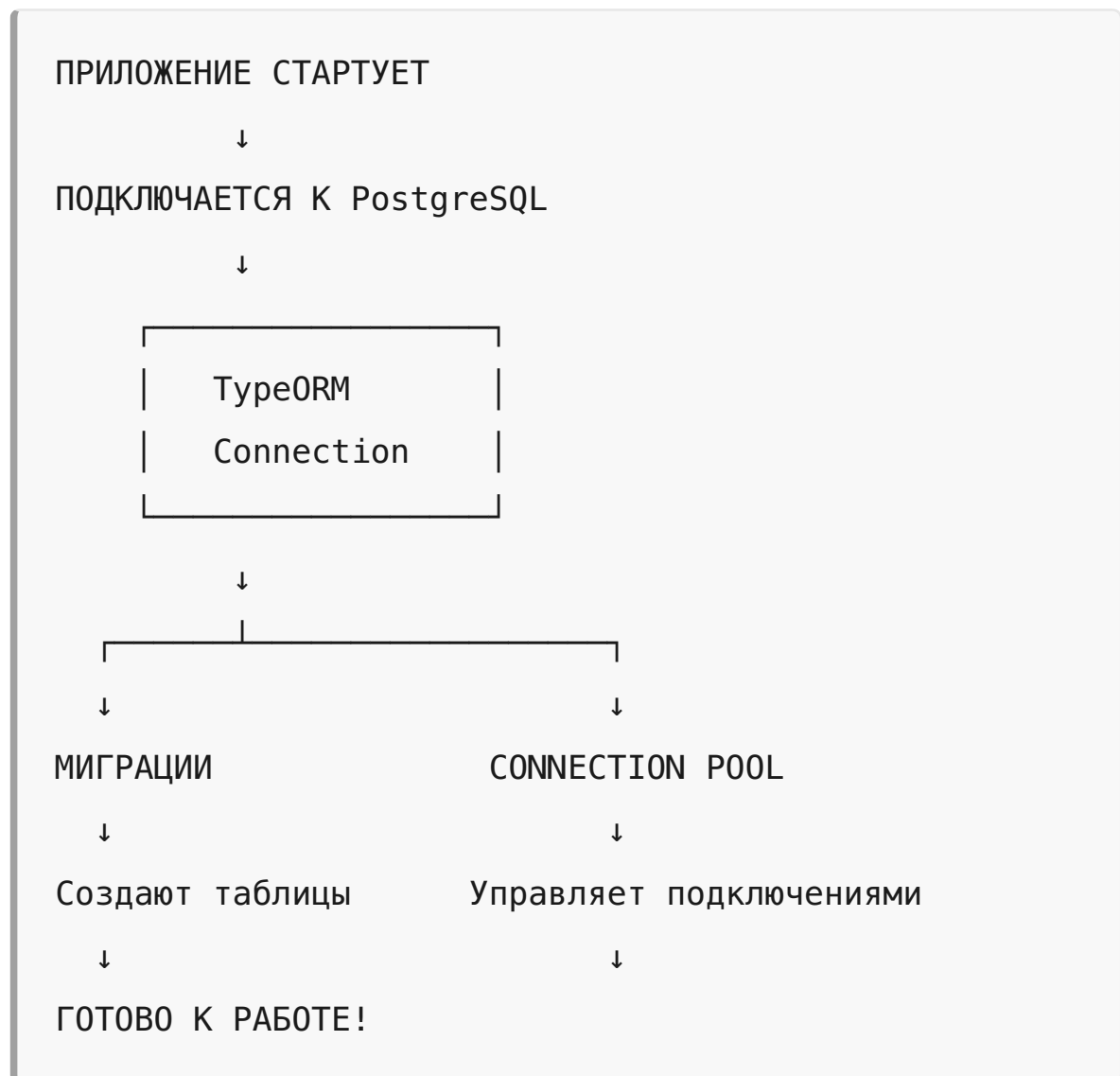
ЧТО ЭТО?

Управление подключениями к PostgreSQL, миграциями и транзакциями

ДЛЯ ЧЕГО НУЖЕН?

Это основа всего - здесь хранятся BCE данные системы.

КАК РАБОТАЕТ?



ЧТО УМЕЕТ?

- Подключаться к PostgreSQL
- Автоматически запускать миграции
- Управлять транзакциями
- Connection pooling (оптимизация)
- Логировать SQL запросы
- Оптимизировать производительность

ИСПОЛЬЗУЕТСЯ В:

- **Kanban Management** → хранение задач

- **AI Agent** → хранение агентов
 - **Flow Management** → хранение флоу
 - **Jira Integration** → настройки подключения
 - **AI Reporting** → данные для отчётов
 - **Photo Analysis** → результаты анализа
 - **Notifications** → уведомления
 - **Context Management** → контекст
-

⚡ 10. CACHE MANAGEMENT

ЧТО ЭТО?

Управление кэшированием через Redis для повышения производительности

ДЛЯ ЧЕГО НУЖЕН?

Вместо того чтобы каждый раз лезть в базу → берём из кэша (в 100x быстрее!)

КАК РАБОТАЕТ?

ЗАПРОС ДАННЫХ



ЧТО УМЕЕТ?

- In-memory кэширование (Redis)
- Автоматическая инвалидация кэша
- TTL управление (время жизни)
- Разные стратегии кэширования
- Мониторинг использования кэша
- Автоматическая очистка старых данных

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ:


```
// Кэшируем задачи на 5 минут
await cache.set('tasks:column:1', tasks, { ttl: 300
});

// Получаем из кэша
const cached = await cache.get('tasks:column:1');

// Инвалидируем при изменении
await cache.del('tasks:column:1');
```

ИСПОЛЬЗУЕТСЯ В:

- **Kanban** → кэширование частых задач
- **AI Agent** → кэширование результатов
- **Flow** → кэширование флоу
- **Jira** → кэширование данных из Jira
- **Reporting** → кэширование отчётов
- **Photo** → кэширование анализа

11. QUEUE MANAGEMENT

ЧТО ЭТО?

Управление очередями через BullMQ для асинхронной обработки задач

ДЛЯ ЧЕГО НУЖЕН?

Долгие операции (AI анализ, синхронизация) выполняются в фоне, не блокируя систему.

КАК РАБОТАЕТ?

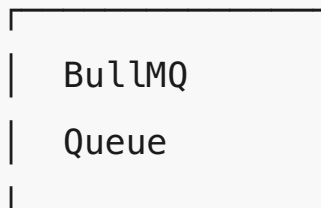
ПОЛЬЗОВАТЕЛЬ ЗАПУСКАЕТ АГЕНТА



API возвращает: "Агент запущен!"



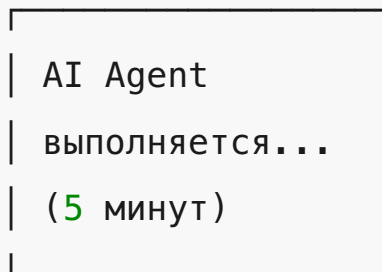
Задача добавляется в очередь



Worker берёт задачу



ВЫПОЛНЯЕТСЯ В ФОНЕ



Результат сохраняется



Отправляется уведомление

ЧТО УМЕЕТ?

Асинхронная обработка задач
Приоритизация (важные - первыми)
Retry механизм (повтор при ошибке)
Dead letter queue (проблемные задачи)
Планирование (cron jobs)
Мониторинг прогресса
Масштабирование workers

ПРИМЕРЫ ЗАДАЧ В ОЧЕРЕДИ:

```
// Добавляем задачу в очередь
await queue.add('run-agent', {
  agentId: '123',
  taskIds: [1, 2, 3]
}, {
  priority: 1, // высокий приоритет
  attempts: 3, // 3 попытки при ошибке
});

// Worker обрабатывает
processor.process('run-agent', async (job) => {
  // Долгая операция
  await runAgent(job.data);
});
```

ИСПОЛЬЗУЕТСЯ В:

- **AI Agent** → выполнение агентов
- **Flow** → выполнение флоу

- **Jira** → синхронизация
 - **Reporting** → генерация отчётов
 - **Photo** → анализ изображений
 - **Notifications** → отправка уведомлений
-

12. CONTEXT MANAGEMENT

ЧТО ЭТО?

Переиспользуемые методы для сбора данных о задачах для AI агентов

ДЛЯ ЧЕГО НУЖЕН?

Вместо того чтобы в каждом сервисе писать код для получения данных из досок → один метод для всех!

КАК РАБОТАЕТ?

БЕЗ CONTEXT MANAGEMENT:

AI Agent Service:

- 50 строк кода для получения данных из Jira

AI Reporting Service:

- 50 строк кода для получения данных из Jira
(копипаста!)

Photo Analysis:

- 50 строк кода... (еще копипаста!)

= 150 строк дублирующегося кода!

С CONTEXT MANAGEMENT:

AI Agent Service:

```
const context = await  
contextMgmt.fetchTaskContext(taskId);
```

AI Reporting Service:

```
const context = await  
contextMgmt.fetchTaskContext(taskId);
```

Photo Analysis:

```
const context = await  
contextMgmt.fetchTaskContext(taskId);
```

= 3 строки! Логика в одном месте!

ЧТО УМЕЕТ?

Получать детали задачи
Получать комментарии
Находить связанные задачи
Получать данные из внешних API
Получать файлы и attachments
Оптимизировать запросы
Кэшировать контекст

ПРИМЕР:

```
// Один вызов – вся информация
const context = await
contextService.fetchTaskContext('task-123');

// Возвращает:
{
  task: { title, description, status, ... },
  comments: [...],
  relatedTasks: [...],
  attachments: [...],
  history: [...]
}
```

ИСПОЛЬЗУЕТСЯ В:

- **AI Agent** → контекст для агентов
- **AI Reporting** → данные для анализа

- **Photo Analysis** → контекст фото
 - **Flow Management** → контекст флоу
-

13. NOTIFICATIONS

ЧТО ЭТО?

Система уведомлений для информирования пользователей о событиях

ДЛЯ ЧЕГО НУЖЕН?

Держит пользователей в курсе: задача создана, агент выполнен, синхронизация завершена.

КАК РАБОТАЕТ?

СОБЫТИЕ ПРОИСХОДИТ

(например, задача создана)



Модуль отправляет событие



Проверяет настройки пользователя



Какие каналы включены?



EMAIL PUSH IN-APP TELEGRAM



Добавляет в очередь (Queue)



Отправляет асинхронно

ЧТО УМЕЕТ?

Отправка уведомлений

Управление подписками

Разные каналы (email, push, in-app)

Шаблоны уведомлений

История уведомлений

Настройки предпочтений пользователя

Группировка уведомлений

ОСНОВНЫЕ ЭНДПОИНТЫ:

- `POST /send-notification` - отправить
- `GET /get-notifications` - получить список
- `POST /mark-as-read` - прочитано
- `GET /get-notification-settings` - настройки
- `PATCH /update-notification-settings` - обновить
- `DELETE /delete-notification` - удалить

ИСПОЛЬЗУЕТСЯ В:

- **Kanban** → уведомления о задачах
- **AI Agent** → завершение работы агента
- **Jira** → результаты синхронизации
- **Flow** → выполнение флоу
- **Reporting** → готовность отчётов

⚙ CONFIGURATION

Все настройки приложения хранятся здесь. Изменил .env файл → поведение приложения изменилось.

ЧТО ЭТО?

Централизованное управление всеми настройками через переменные окружения

ДЛЯ ЧЕГО НУЖНО?

- Разные настройки для DEV и PROD
- Безопасное хранение секретов (API ключи, пароли)
- Легко менять настройки без изменения кода

КАК ЭТО РАБОТАЕТ?

`.env` ФАЙЛ

CONFIG FILES

<code>.env</code> ФАЙЛ		CONFIG FILES
<code>PORT=3000</code>	→	<code>app.config.ts</code>
<code>DB_HOST=localhost</code>	→	<code>database.config.ts</code>
<code>REDIS_HOST=redis</code>	→	<code>cache.config.ts</code>
<code>CLAUDE_API_KEY=xxx</code>	→	<code>claude.config.ts</code>
<code>JIRA_HOST=xxx</code>	→	<code>jira.config.ts</code>

ОСНОВНЫЕ КОНФИГУРАЦИОННЫЕ ФАЙЛЫ:

1. `app.config.ts` - Основные настройки

```
{
  port: 3000,           // Порт приложения
  environment: 'dev',   // dev / prod
  apiPrefix: 'api'      // Префикс для всех
ЭНДПОИНТОВ
}
```

2. database.config.ts - PostgreSQL

```
{
  host: 'localhost',
  port: 5432,
  database: 'kanban_db',
  username: 'postgres',
  password: 'secret'
}
```

3. cache.config.ts - Redis

```
{
  host: 'localhost',
  port: 6379,
  password: null,
  ttl: 300 // время жизни кэша по умолчанию
}
```

4. queue.config.ts - BullMQ

```
{
  redis: {
    host: 'localhost',
    port: 6379
  },
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: 'exponential', delay: 1000 }
  }
}
```

5. claude.config.ts - AI модели

```
{
  apiKey: process.env.CLAUDE_API_KEY,
  model: 'claude-3-sonnet',
  maxTokens: 4096,
  temperature: 0.7
}
```

6. jira.config.ts - Jira

```
{
  host: 'company.atlassian.net',
  email: 'user@company.com',
  apiToken: process.env.JIRA_API_TOKEN
}
```

7. notifications.config.ts - Уведомления

```
{  
  smtp: {  
    host: 'smtp.gmail.com',  
    port: 587,  
    user: 'noreply@app.com'  
  },  
  emailFrom: 'Kanban AI <noreply@app.com>'  
}
```

БЕЗОПАСНОСТЬ:

`.env` файл в `.gitignore` (не коммитится)

На продакшене используются environment variables

Валидация всех переменных при старте

Никогда не храним секреты в коде

DATABASE ENTITIES

TypeORM сущности - это схема базы данных. Каждая entity = таблица в PostgreSQL.

ЧТО ЭТО?

Определение структуры базы данных через TypeScript
классы

ДЛЯ ЧЕГО НУЖНО?

- Типизация данных из БД
- Автоматическое создание таблиц
- Связи между таблицами
- Валидация данных

КАК ЭТО РАБОТАЕТ?

```

@Entity('agents') // ← Создаст таблицу "agents"
export class Agent {
    @PrimaryGeneratedColumn('uuid')
    id: string; // ← PRIMARY KEY

    @Column()
    name: string; // ← VARCHAR

    @Column({ type: 'enum', enum: AgentType })
    type: AgentType; // ← ENUM

    @Column({ type: 'jsonb', nullable: true })
    config: Record<string, any>; // ← JSON

    @OneToMany(() => AgentInstruction, instr =>
instr.agent)
    instructions: AgentInstruction[]; // ← СВЯЗЬ 1-
КО-МНОГИМ

    @CreateDateColumn()
    createdAt: Date; // ← Автоматически

    @UpdateDateColumn()
    updatedAt: Date; // ← Автоматически
}

```

ОСНОВНЫЕ ENTITIES:

1. agent.entity.ts - AI агенты

Таблица: agents

id	UUID
name	VARCHAR
type	ENUM (task_creator, analyzer, custom)
status	ENUM (idle, running, completed, failed)
config	JSONB
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

2. agent-instruction.entity.ts - Инструкции для агентов

Таблица: agent_instructions

id	UUID
agentId	UUID (FK → agents.id)
instruction	TEXT
order	INTEGER
createdAt	TIMESTAMP

3. flow.entity.ts - Визуальные флоу

Таблица: flows

id	UUID
name	VARCHAR
blocks	JSONB (массив блоков)
edges	JSONB (связи между блоками)
status	ENUM (draft, active, archived)
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

4. task-history.entity.ts - История изменений задач

Таблица: task_history

id	UUID
taskId	UUID
changes	JSONB (что изменилось)
userId	UUID
timestamp	TIMESTAMP

5. board-integration.entity.ts - Интеграции с досками

Таблица: board_integrations

id	UUID
type	ENUM (jira, trello, asana, monday)
credentials	JSONB (зашифрованные)
syncSettings	JSONB (настройки синхронизации)
lastSyncAt	TIMESTAMP
createdAt	TIMESTAMP

6. notification-log.entity.ts - Лог уведомлений

Таблица: notification_logs

id	UUID
userId	UUID
message	TEXT
status	ENUM (pending, sent, failed)
channel	ENUM (email, push, in-app)
sentAt	TIMESTAMP

СВЯЗИ МЕЖДУ ТАБЛИЦАМИ:

Agent (1) — (N) AgentInstruction

|
└ (N) TaskHistory

Flow (1) — (N) FlowBlock

BoardIntegration (1) — (N) Task

User (1) — (N) NotificationLog

АВТОМАТИЧЕСКИЕ ФИЧИ:

Timestamps - `createdAt` , `updatedAt` создаются
автоматически

UUID - уникальные ID генерируются автоматически

Soft Delete - можно удалять “мягко” (не из БД)

Индексы - для быстрого поиска

Валидация - TypeORM проверяет типы

NESTJS MODULES

Модули управляют зависимостями между компонентами. Это как “упаковки” кода.

ЧТО ЭТО?

Организация кода через модули с Dependency Injection

ДЛЯ ЧЕГО НУЖНО?

- Разделение кода на логические части
- Управление зависимостями
- Переиспользование кода
- Ленивая загрузка модулей

КАК ЭТО РАБОТАЕТ?

```
@Module({
  imports: [
    // Какие модули импортируем
    DatabaseModule,
    CacheModule,
  ],
  controllers: [
    // API контроллеры
    AgentController,
  ],
  providers: [
    // Сервисы (бизнес-логика)
    AgentService,
  ],
  exports: [
    // Что экспортируем для других модулей
    AgentService,
  ],
})
export class AgentModule {}
```

ТИПЫ МОДУЛЕЙ:

1. Feature Modules - Бизнес-логика

<code>ai-agent.module.ts</code>	→ AI агенты
<code>flow-management.module.ts</code>	→ Flow builder
<code>jira-integration.module.ts</code>	→ Jira интеграция
<code>kanban-management.module.ts</code>	→ Канбан
<code>ai-reporting.module.ts</code>	→ Аналитика
<code>photo-analysis.module.ts</code>	→ Анализ фото
<code>notifications.module.ts</code>	→ Уведомления

2. Infrastructure Modules - Инфраструктура

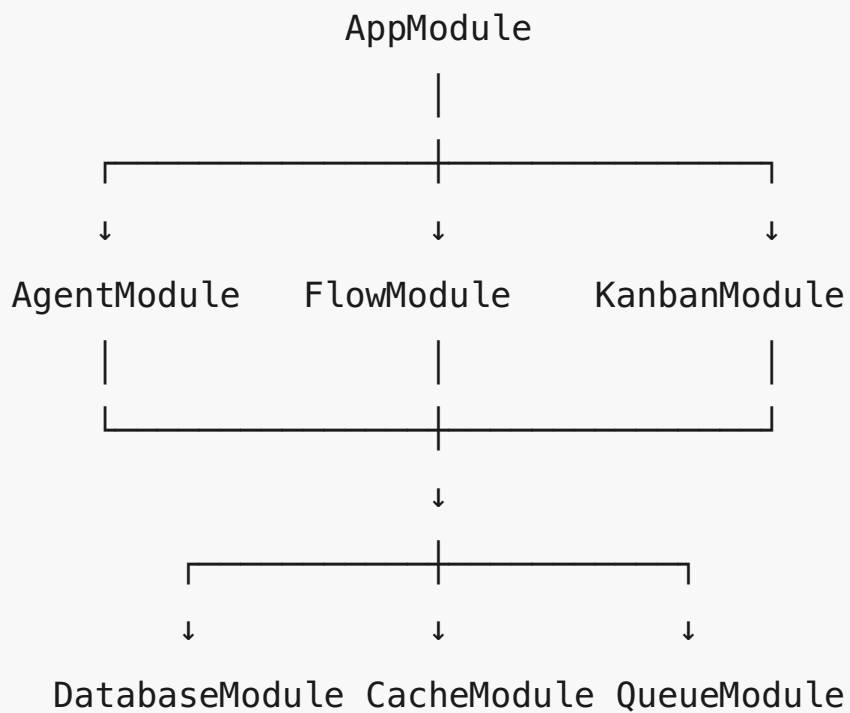
<code>database.module.ts</code>	→ TypeORM настройка
<code>database-management.module.ts</code>	→ Управление БД
<code>cache-management.module.ts</code>	→ Redis кэш
<code>queue-management.module.ts</code>	→ BullMQ очереди
<code>context-management.module.ts</code>	→ AI контекст

3. Dynamic Modules - Конфигурируемые

```
@Module({})
export class CacheModule {
    static forRoot(options: CacheOptions):
DynamicModule {
    return {
        module: CacheModule,
        providers: [
            { provide: 'CACHE_OPTIONS', useValue:
options },
            CacheService,
        ],
        exports: [CacheService],
    };
}
}

// Использование:
CacheModule.forRoot({ ttl: 300 })
```

ГРАФ ЗАВИСИМОСТЕЙ:



DEPENDENCY INJECTION:

```
@Injectable()
export class AgentService {
  constructor(
    // DI автоматически внедряет зависимости
    private readonly database: DatabaseService,
    private readonly cache: CacheService,
    private readonly queue: QueueService,
  ) {}
}
```

TYPESCRIPT TYPES

Типы и интерфейсы для строгой типизации всего приложения.

ЧТО ЭТО?

Централизованное хранение всех TypeScript типов

ДЛЯ ЧЕГО НУЖНО?

- Type-safety (ловим ошибки на этапе разработки)
- Автокомплит в IDE
- Самодокументирование кода
- Легкий рефакторинг

ОСНОВНЫЕ ТИПЫ:

1. Agent Types

```
interface Agent {
  id: string;
  name: string;
  type: AgentType;
  status: AgentStatus;
  config: AgentConfig;
  instructions: AgentInstruction[];
  createdAt: Date;
}

enum AgentType {
  TASK_CREATOR = 'task_creator',
  TASK_ANALYZER = 'task_analyzer',
  CUSTOM = 'custom',
}

enum AgentStatus {
  IDLE = 'idle',
  RUNNING = 'running',
  COMPLETED = 'completed',
  FAILED = 'failed',
}
```

2. Flow Types

```
type FlowBlock = {  
  id: string;  
  type: FlowBlockType;  
  position: { x: number; y: number };  
  data: FlowBlockData;  
};  
  
type FlowEdge = {  
  id: string;  
  source: string; // ID блока-источника  
  target: string; // ID блока-назначения  
  type?: 'default' | 'conditional';  
};  
  
type FlowDefinition = {  
  blocks: FlowBlock[];  
  edges: FlowEdge[];  
};
```

3. Board Integration Types

```
interface BoardConnection {
    id: string;
    type: BoardType;
    credentials: BoardCredentials;
    syncSettings: SyncSettings;
}

enum BoardType {
    JIRA = 'jira',
    TRELLO = 'trello',
    ASANA = 'asana',
    MONDAY = 'monday',
}

interface IBoardService {
    connect(credentials: any): Promise<void>;
    disconnect(): Promise<void>;
    getTasks(filters?: any): Promise<Task[]>;
    createTask(data: CreateTaskDto): Promise<Task>;
    syncTasks(): Promise<SyncResult>;
}
```

4. Context Types

```
interface Context {  
    task: Task;  
    comments: Comment[];  
    relatedTasks: Task[];  
    attachments: Attachment[];  
    history: TaskHistory[];  
}  
  
interface ContextWindow {  
    maxTokens: number;  
    currentTokens: number;  
    content: string;  
}
```

ПРЕИМУЩЕСТВА ТИПИЗАЦИИ:

```
//      БЕЗ ТИПОВ:
function createAgent(data) {
  // Что в data? Неизвестно!
  // Можно передать что угодно
  return service.create(data);
}

//      С ТИПАМИ:
function createAgent(data: CreateAgentDto):
Promise<Agent> {
  // IDE подсказывает все поля
  // TypeScript проверяет типы
  // Ошибки видны сразу
  return service.create(data);
}
```

SHARED SERVICES

Переиспользуемые сервисы для интеграций с внешними системами.

ЧТО ЭТО?

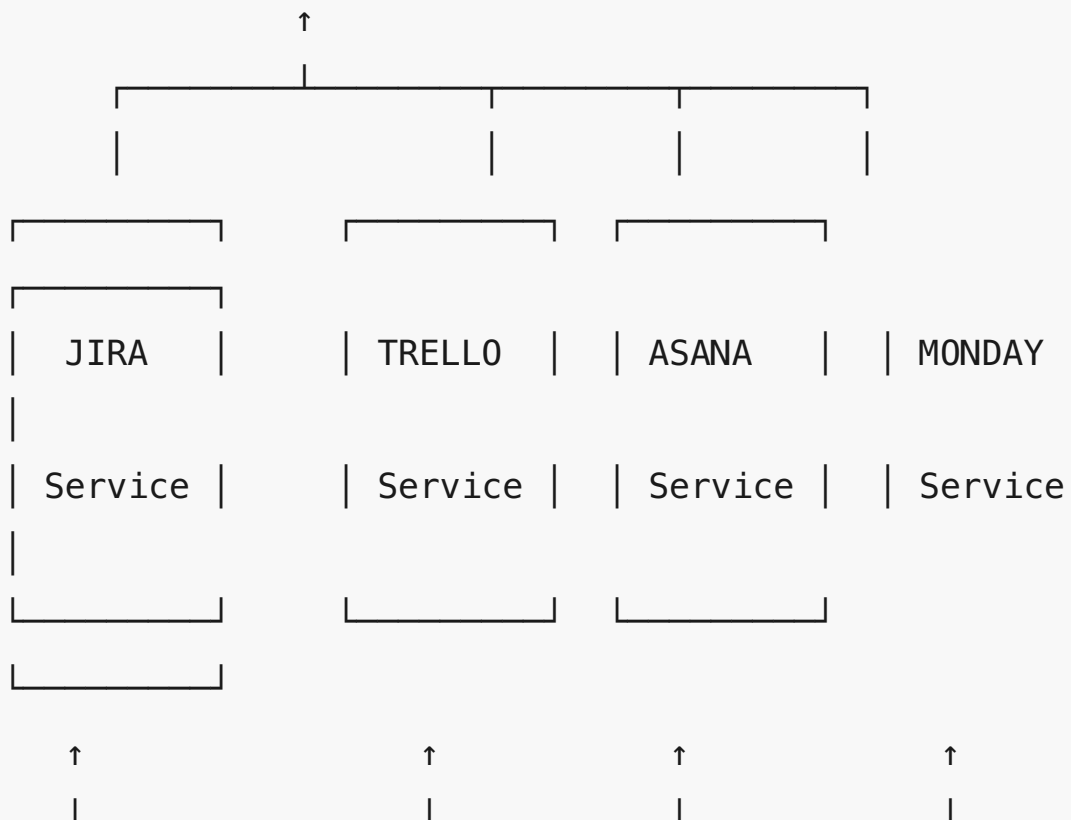
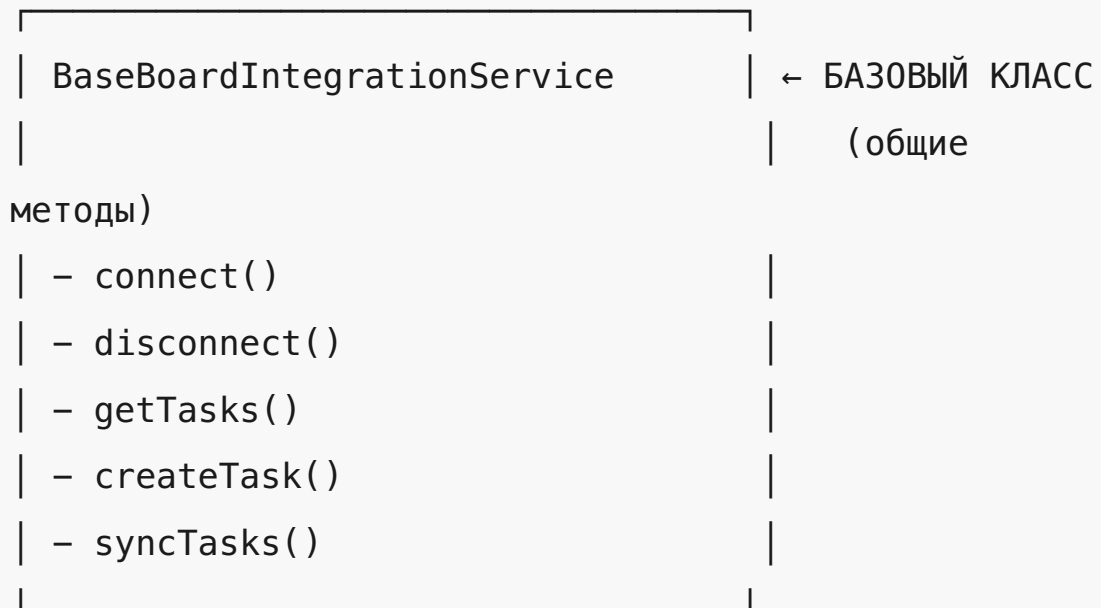
Базовые классы и фабрики для работы с разными досками задач

ДЛЯ ЧЕГО НУЖНО?

Вместо того чтобы писать отдельный код для Jira, Trello, Asana → пишем один базовый класс!

КАК ЭТО РАБОТАЕТ?

БАЗОВЫЙ КЛАСС (ПАТТЕРН НАСЛЕДОВАНИЯ):



Реализуют одинаковые методы!

БАЗОВЫЙ КЛАСС:

```
export abstract class BaseBoardIntegrationService {  
    // Абстрактные методы (должны реализовать  
наследники)  
    abstract connect(credentials: any): Promise<void>;  
    abstract disconnect(): Promise<void>;  
    abstract getTasks(filters?: any): Promise<Task[]>;  
    abstract createTask(data: CreateTaskDto):  
Promise<Task>;  
  
    // Общие методы (работают для всех)  
    protected validateCredentials(credentials: any):  
void {  
        if (!credentials) throw new Error('Credentials  
required');  
    }  
  
    protected mapToInternalTask(externalTask: any):  
Task {  
        // Конвертация внешней задачи в нашу Task  
        return {  
            id: externalTask.id,  
            title: externalTask.summary ||  
externalTask.name,  
            description: externalTask.description,  
            status: this.mapStatus(externalTask.status),  
        };  
    }  
}
```

```
}  
}
```

ФАБРИКА:

```
@Injectable()  
export class BoardIntegrationFactory {  
  create(type: BoardType):  
    BaseBoardIntegrationService {  
    switch (type) {  
      case BoardType.JIRA:  
        return new JiraBoardService();  
      case BoardType.TRELL0:  
        return new TrelloBoardService();  
      case BoardType.ASANA:  
        return new AsanaBoardService();  
      default:  
        throw new Error(`Unknown board type:  
        ${type}`);  
    }  
  }  
}
```

JIRA РЕАЛИЗАЦИЯ:

```
export class JiraBoardService extends
BaseBoardIntegrationService {
  private client: JiraClient;

  async connect(credentials: JiraCredentials):
Promise<void> {
    this.validateCredentials(credentials);
    this.client = new JiraClient(credentials);
    await this.client.testConnection();
  }

  async getTasks(filters?: JiraFilters):
Promise<Task[]> {
    const issues = await
this.client.getIssues(filters);
    return issues.map(issue =>
this.mapToInternalTask(issue));
  }

  // ... остальные методы
}
```

ИСПОЛЬЗОВАНИЕ:

```
// Создаём нужный адаптер
const boardService = factory.create(BoardType.JIRA);

// Подключаемся
await boardService.connect(credentials);

// Получаем задачи (одинаково для всех досок!)
const tasks = await boardService.getTasks();

// Создаём задачу (одинаково для всех досок!)
await boardService.createTask({ title: 'New task'
});
```

ПРЕИМУЩЕСТВА:

DRY - Don't Repeat Yourself (код не дублируется)

Легко добавлять новые интеграции - наследуй базовый класс

Единый интерфейс - работаешь одинаково со всеми досками

Типизация - TypeScript проверяет всё

КАК ВСЁ РАБОТАЕТ ВМЕСТЕ

Полная картина: от запроса пользователя до ответа.

СЦЕНАРИЙ 1: Создание и выполнение AI агента

1. ПОЛЬЗОВАТЕЛЬ ОТПРАВЛЯЕТ ЗАПРОС

↓

POST /api/ai-agent/create-agent

```
{ name: "Task Analyzer", type: "analyzer" }
```

2. NESTJS РОУТИНГ

↓

AgentController.createAgent()

3. VALIDATION

↓

CreateAgentDto проверяется (TypeScript + class-validator)

4. БИЗНЕС-ЛОГИКА

↓

AgentService.create(dto)

└─ Проверяет права доступа

└─ Валидирует конфигурацию

└─ Создает агента

5. СОХРАНЕНИЕ В БД

↓

DatabaseService.save(agent)

└─ TypeORM создает запись в таблице agents

↳ Возвращает Agent entity

6. КЭШИРОВАНИЕ

↓

```
CacheService.set('agent:123', agent, { ttl: 300 })
```

↳ Сохраняет в Redis

7. ОТВЕТ ПОЛЬЗОВАТЕЛЮ

↓

```
{ id: '123', name: 'Task Analyzer', status: 'idle' }
```

ЗАПУСК АГЕНТА:

8. ПОЛЬЗОВАТЕЛЬ ЗАПУСКАЕТ

↓

POST /api/ai-agent/run-agent/123

9. ДОБАВЛЕНИЕ В ОЧЕРЕДЬ

↓

```
QueueService.add('agent-execution', { agentId: '123' })
```

↳ Задача добавляется в BullMQ

10. НЕМЕДЛЕННЫЙ ОТВЕТ

↓
`{ status: 'queued', message: 'Agent started' }`

11. WORKER БЕРЁТ ЗАДАЧУ (в фоне)

↓
`AgentWorker.process(job)`

12. ПОЛУЧЕНИЕ КОНТЕКСТА

↓
`ContextService.fetchTaskContext(taskIds)`
├→ Получает задачи из Kanban
├→ Получает комментарии
├→ Получает связанные задачи
└→ Формирует контекст для AI

13. ВЫЗОВ AI

↓
`ClaudeService.chat(prompt, context)`
└→ Отправляет в Claude API

14. ОБРАБОТКА РЕЗУЛЬТАТА

↓
`KanbanService.updateTasks(aiResults)`
└→ Обновляет задачи на основе AI ответа

15. СОХРАНЕНИЕ РЕЗУЛЬТАТА

↓
`DatabaseService.save(agentResult)`
└→ Сохраняет в БД

16. УВЕДОМЛЕНИЕ

↓

```
NotificationService.send(userId, 'Agent  
completed')
```

└─ Отправляет через очередь

СЦЕНАРИЙ 2: Синхронизация с Jira

1. ПОЛЬЗОВАТЕЛЬ ЗАПУСКАЕТ СИНХРОНИЗАЦИЮ

↓

```
POST /api/jira/sync-with-jira
```

2. ПОЛУЧЕНИЕ АДАПТЕРА

↓

```
BoardIntegrationFactory.create(BoardType.JIRA)
```

└─ Создаёт JiraBoardService

3. ПОДКЛЮЧЕНИЕ К JIRA

↓

```
JiraBoardService.connect(credentials)
```

└─ Берёт credentials из Database

└─ Создаёт Jira клиент

└─ Проверяет подключение

4. ПОЛУЧЕНИЕ ЗАДАЧ ИЗ JIRA

↓

`JiraBoardService.getTasks(filters)`

↳ Вызывает Jira API

5. ПРОВЕРКА КЭША

↓

`CacheService.get('jira:tasks:last-sync')`

↳ Проверяет, что изменилось с последней синхронизации

6. МАППИНГ ДАННЫХ

↓

`BaseBoardIntegrationService.mapToInternalTask(jiraIssue)`

↳ Конвертирует Jira Issue → Task

↳ Маппит статусы (In Progress → В работе)

↳ Сохраняет связь с Jira

7. СОХРАНЕНИЕ В КАНБАН

↓

`KanbanService.createOrUpdateTasks(tasks)`

↳ Проверяет, существует ли задача

↳ Создает новую или обновляет

↳ Сохраняет историю изменений

8. ОБРАТНАЯ СИНХРОНИЗАЦИЯ

↓

`KanbanService.getModifiedTasks(since: lastSync)`

↳ Находит задачи, изменённые в Канбане

9. ЭКСПОРТ В JIRA

↓

```
JiraBoardService.updateTasks(modifiedTasks)
```

↳ Обновляет задачи в Jira

10. УВЕДОМЛЕНИЕ

↓

```
NotificationService.send('Sync completed: 15  
imported, 3 exported')
```

СЦЕНАРИЙ 3: Flow Builder → AI Agent

1. СОЗДАНИЕ ФЛОУ (на фронтенде)

↓

Пользователь строит визуальный флоу:

```
[START] → [GET TASKS] → [ANALYZE] → [UPDATE] →  
[END]
```

2. СОХРАНЕНИЕ ФЛОУ

↓

```
POST /api/flow/create-flow
```

```
{  
  name: "Auto Task Analyzer",  
  blocks: [...],  
  edges: [...]  
}
```

3. ВАЛИДАЦИЯ СТРУКТУРЫ

↓

`FlowService.validate(flowDefinition)`

└─> Проверяет, что есть START и END блоки

└─> Проверяет связи между блоками

└─> Проверяет нет ли циклов

└─> Проверяет типы блоков

4. СОХРАНЕНИЕ

↓

`DatabaseService.save(flow)`

└─> Сохраняет в таблицу flows (JSONB)

КОНВЕРТАЦИЯ В АГЕНТА:

5. ЗАПРОС КОНВЕРТАЦИИ

↓

`POST /api/flow/convert-flow-to-agent/123`

6. ПОЛУЧЕНИЕ ФЛОУ

↓

`FlowService.getFlow(123)`

└─> Берёт из кэша или БД

7. АНАЛИЗ БЛОКОВ

↓

`FlowConversionService.analyzeBlocks(flow.blocks)`

└─> Определяет последовательность выполнения

└─> Находит условные блоки (if/else)

└─> Строит граф выполнения

8. ГЕНЕРАЦИЯ ПРОМПТА

↓

`FlowConversionService.generatePrompt(blocks)`

Пример результата:

"Ты – AI агент для анализа задач.

Шаг 1: Получи все задачи из колонки 'To Do'

Шаг 2: Для каждой задачи:

- Проанализируй title и description
- Определи категорию (bug, feature, task)
- Оцени приоритет (low, medium, high)

Шаг 3: Обнови задачи с новыми тегами

Шаг 4: Верни статистику"

9. СОЗДАНИЕ АГЕНТА

↓

`AgentService.createFromFlow(flowId, prompt)`

└─> Создает Agent entity

└─> Создает AgentInstructions

└─> Сохраняет связь с Flow

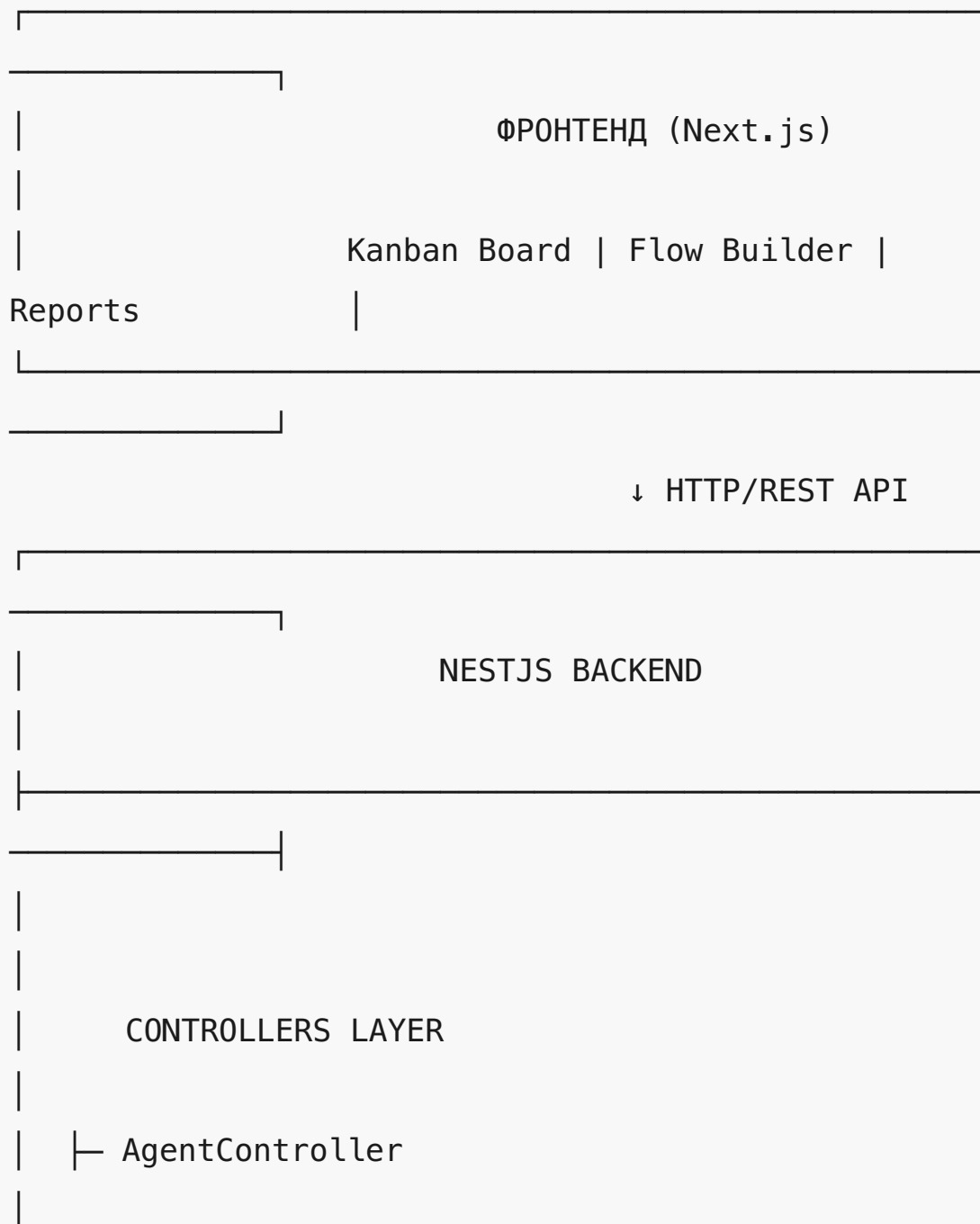
└─> Возвращает готового агента

10. ГОТОВО!

↓

Агент можно сразу запускать через `/run-agent`

ПОЛНАЯ АРХИТЕКТУРНАЯ ДИАГРАММА



| └─ FlowController

| └─ KanbanController

| └─ JiraController

| └─ ... остальные

| ↓

| SERVICES LAYER (Бизнес-логика)

| └─ AgentService

| └─ FlowService

| └─ KanbanService

| └─ JiraIntegrationService

| └─ AIReportingService

| └─ PhotoAnalysisService

| └─ ... остальные

| ↓

| INFRASTRUCTURE LAYER

| DatabaseService → PostgreSQL

| CacheService → Redis

| QueueService → BullMQ

| ContextService → AI Context

| NotificationService → Email/Push

↓

ENTITIES LAYER

| TypeORM Entities → Database Tables

↓

ВНЕШНИЕ СИСТЕМЫ

| PostgreSQL – Основная база данных

✂ Redis – Кэш и очереди

Claude AI – Искусственный интеллект

Jira API – Синхронизация задач

SMTP – Отправка email

ОСНОВНЫЕ ПОТОКИ ДАННЫХ

1. USER REQUEST → Controller → Service → Database → Response

2. ASYNC TASK → Queue → Worker → AI → Database → Notification

3. JIRA SYNC → Adapter → Mapper → Kanban → Database → Cache

4. FLOW → Converter → Agent → Queue → Execution → Result

ЗАКЛЮЧЕНИЕ

ЧТО МЫ ПОСТРОИЛИ:

13 модулей, каждый с чёткой задачей:

Feature-модули (бизнес-логика):

1. Kanban Management - управление задачами
2. AI Agent - умные агенты
3. Flow Management - визуальный конструктор
4. Jira Integration - синхронизация с Jira
5. AI Reporting - аналитика через AI
6. Photo Analysis - анализ изображений
7. Board Integrations - другие интеграции
8. Flow Conversion - флоу → агенты

✂ Infrastructure-модули (фундамент):

9. Database Management - PostgreSQL
10. Cache Management - Redis
11. Queue Management - BullMQ
12. Context Management - AI контекст
13. Notifications - уведомления

Дополнительно:

- ⚙ Configuration - настройки
- Database Entities - схема БД
- NestJS Modules - DI

- TypeScript Types - типизация
- Shared Services - переиспользуемый код

КЛЮЧЕВЫЕ ПРИНЦИПЫ:

Модульность - каждый модуль независим

Типизация - TypeScript везде

Асинхронность - долгие операции в очереди

Кэширование - быстрый доступ к данным

DRY - код не дублируется

SOLID - правильная архитектура

ПРОИЗВОДИТЕЛЬНОСТЬ:

- ✂ Redis кэш → 100x быстрее чем БД
- BullMQ → асинхронная обработка
- PostgreSQL → надёжное хранилище
- Connection pooling → оптимизация запросов

БЕЗОПАСНОСТЬ:

- Credentials в .env (не в коде!)
 - Валидация всех входных данных
 - API токены для внешних сервисов
 - Логирование всех операций
-